腾讯混元AI Infra如何优化Hy3 Preview：一次大模型推理性能提升的技术拆解

原创 腾讯程序员 腾讯技术工程 Jun 26, 2026, 2:36 AM 广东 532 listened



作者：混元AI Infra推理团队

> 本文将从算子优化与融合、并行策略、多级缓存、MTP和异步调度优化、量化与稀疏五大维度，逐一剖析各项技术的设计思路、核心算法与实测收益，全面揭示 Hy3 preview 模型在 Hopper 卡上从算子到系统的极致性能优化实践。

一、背景和优化成果

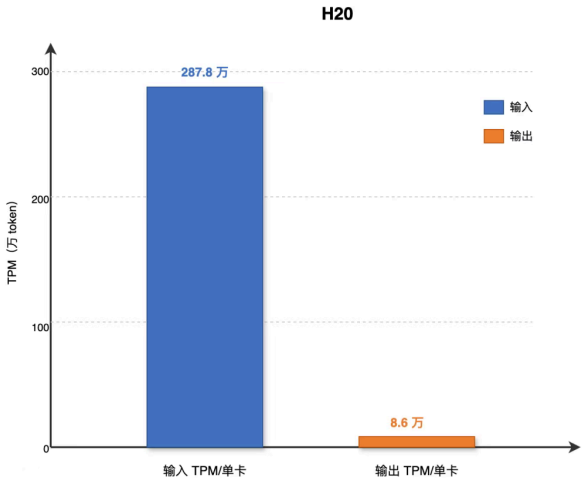
随着大语言模型向千亿参数、百万级上下文加速演进，推理效率已成为模型规模化落地的决定性瓶颈。更大的模型、更长的序列、更复杂的 MoE 稀疏架构，在带来能力跃升的同时，也对算力、显存与通信提出了前所未有的挑战。

混元Hy3 preview作为腾讯新一代旗舰大模型，采用 GQA + MoE 混合架构，原生支持 256K 超长上下文，面向 Agent、Coding 等场景。然而主部署卡 NVIDIA Hopper卡 相比业界主流 Blackwell 系列卡存在算力极低、显存紧凑、缺乏超节点互联等劣势，这意味着必须在极其有限的硬件和长上下文场景下，将推理性能优化至极致方能满足 SLO 约束并实现成本最优。

面对这一系统性挑战，混元 AI Infra 推理团队对 Hy3 preview 推理全栈进行了深度优化。本文将从**算子优化与融合、并行策略、多级缓存、MTP和异步调度优化、量化与稀疏**五大维度，逐一剖析各项技术的设计思路、核心算法与实测收益，全面揭示 Hy3 preview 模型在Hopper卡上从算子到系统的极致性能优化实践。

优化成果

- 测试数据集：5000条真实数据(最大输入192k，平均输入68k；最大输出64k， 平均输出0.9k；缓存理论命中80%)
- 测试机器： **Hopper架构-96G**
- 限制**50ms TPOP，4s TTFT**
- 精度： W8A8C8

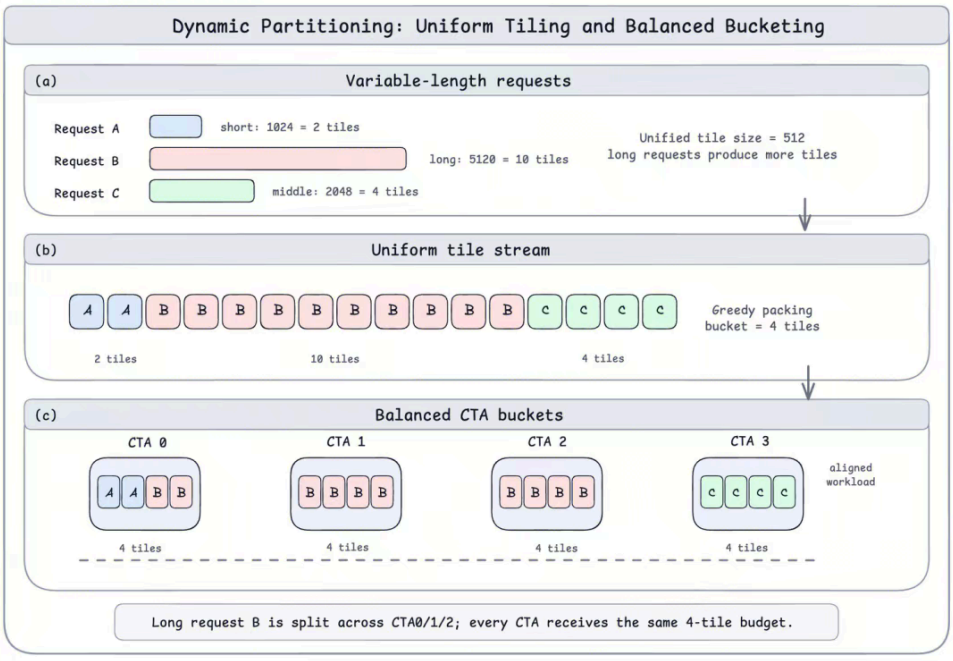


二、性能优化方案

2.1 算子优化

针对Hopper架构和Hy3 preview模型，我们深度优化Attention、MoE、Rope、Router、Sampler和通信算子，并开源在**HPC-Ops仓库**。

2.1.1 Attention：动态调度负载均衡，根治推理长尾延迟



问题

线上推理存在请求长度实时波动、batch 内长短请求混杂等特征， 传统静态 split-kv 需要在长序列吞吐和短序列开销之间做固定权衡。长序列需要更大的 split-kv 才能充分并行，短序列则只需要少量拆分，固定策略很难两头兼顾。

动态调度方案

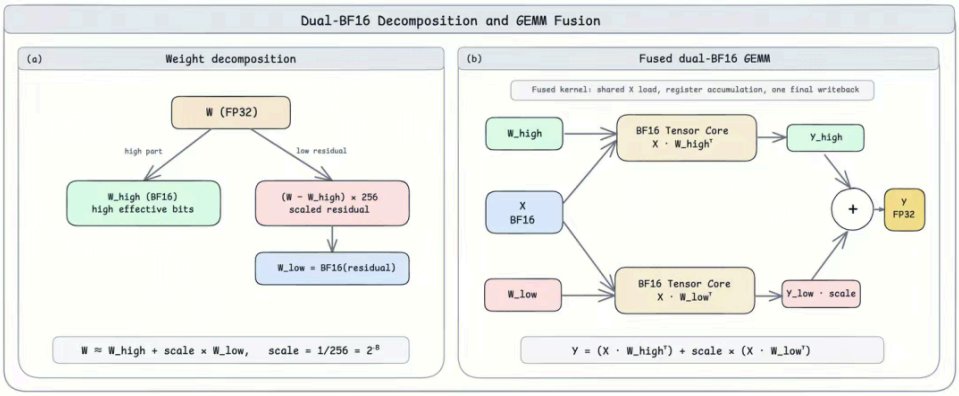
- 所有推理请求按统一Tile粒度拆分，依据全局Tile总量均衡分配各CTA任务规模
- 再通过贪心装桶算法实现Tile任务极致均分，从源头杜绝计算单元负载失衡问题

- 在执行链路中，Task Assign 模块会在每次推理前生成专属任务映射表，各层Attention Kernel依据映射表精准领取并执行对应任务
- 最终由Combine Kernel统一合并split-kv计算结果，实现全流程负载均衡、高效协同运算。

性能收益

- 单 batch 长文本场景下单算子最高加速 **2.95x**
- 混合长度 batch 场景下加速 **1.59x~1.76x**

2.1.2 Router GEMM：双BF16重构FP32计算，兼顾高精度与高性能



问题

在 MoE 路由及稀疏 Attention 等数值敏感模块中，传统的 BF16 激活 × FP32 权重 计算面临效率与精度的两难抉择：若将权重降级，会显著损耗模型精度；若将激活升频至 FP32/TF32，则需引入逐元素类型转换的额外开销，且受限于 CUDA Core 较低的算力带宽，硬件利用率极低。

方案

1. 离线阶段将 FP32 权重拆分为高位 BF16 与低位残差 BF16 两组张量：**W ≈ W_high + scale × W_low**（scale = 1/256，对齐 BF16 的 8 位尾数）。
2. 推理阶段执行两次 BF16 GEMM 并做线性组合，激活值全程保持 BF16，无需类型转换，均运行在 BF16 Tensor Core 上。
3. 实现上，双路计算融合至单一 Kernel：输入数据仅搬运一次，双寄存器累加器分别缓存两路中间结果，Epilogue 阶段经一次 FFMA 修正出高精度结果写出显存，全程无 HBM 往返，无冗余开销。

性能收益：N=192、K=4096 规格，在M=2~4096范围 相比FP32(cuBLAS)实现 有 **2.86x ～ 3.22x** 加速

2.1.3 FusedMoE：全算子流水线重构，极致压缩推理开销

重构方案

HPC-Ops 对 MoE 完整推理链路进行深度融合与执行逻辑重排，将五大核心阶段整合为一体化执行链路：

1. **路由与索引预处理**：基于共享内存分块统计，为每个专家预留连续显存输出区间，大幅降低大规模 Token 场景下的索引构建成本。
2. **Gate-Up GEMM**：通过路由索引直接读取原始输入，省去显式 Gather 搬运；取消 Warp Specialization，由同一 Warp Group 完成搬运与计算，将访存掩盖逻辑由 CTA 内软件流水升级为跨 CTA 硬件调度，显著提升 SM 驻留密度与资源利用率。
3. **激活量化 + Down GEMM**：量化结果按专家维度紧凑写入显存，确保 Down GEMM 高效顺序访存。
4. **Top-K 加权聚合**：在推理末端完成加权求和，全程无额外 HBM 往返。
5. **PDL 无气泡串联**：全流程通过 PDL 技术构建无气泡执行链路，彻底消除频繁 Kernel 启动开销。

性能收益

- TP=8 / EP=1 场景：相比 vLLM CUTLASS、vLLM Triton、SGLang 获得 **1.5x – 1.6x** 加速
- TP=1 / EP=8 场景：获得 **1.2x – 1.5x** 加速

2.2 算子融合

2.2.1 Fused Rope+Norm+[Hadamard]+Quant+Store KV

在 QKV Projection 之后，存在连续的 Element-wise 算子链（Rope、RMSNorm、Hadamard 积、量化、KV Cache 写入）。由于各算子计算量极小且算力强度低，频繁启动 Kernel 并反复读写 HBM 导致严重的 **访存带宽受限**，成为 Prefill 阶段不可忽视的延迟来源。

我们通过算子深度融合，将上述 5 个算子重构为单一的微型流水线 Kernel。数据从 HBM 载入寄存器后，在片上完成全链路变换，最终仅写回一次结果，将多次 HBM 往返压减至最低。

- **寄存器级数据流转**：全流程采用寄存器暂存中间结果，彻底消除中间变量（如 Norm 后、Rope 后）的 HBM 存取开销。
- **在线量化与KVCache存储**：在写入 KV Cache 前完成在线量化（Quant），直接以低比特格式写入显存，进一步压缩写出带宽。

性能收益：融合算子加速约**5x**

2.2.2 Fused AllReduce+Norm+Add

针对张量并行场景下，通信、残差计算、归一化拆分执行导致的性能损耗，联合腾讯网络平台部，创新实现通信、残差计算与归一化的全链路融合，封装为 NVLink 原生一体化操作：RMSNorm(AllReduce(x) + residual, weight)。基于 CUDA 多播与 P2P 技术，支持 BF16 及单机多卡部署，采用高效 Two-shot 策略。

- 高吞吐版本（fuse_allreduce_rmsnorm_high_throughput）依托NVSwitch多播机制完成归约计算，适配大规模 Token 的 Prefill 预处理场景；
- 低延迟版本（fuse_allreduce_rmsnorm_low_latency）基于Lamport P2P机制，通过PDL实现双Kernel重叠执行，适配小批量Token的Decode推理场景。

性能收益

- 覆盖8~32k tokens 场景
- 相比NCCL与FlashInfer同类路径，实测最高加速**1.68x**

2.2.3 采样融合算子

传统采样后处理链路由十余个零碎 Kernel 串联实现（重复惩罚、温度缩放、Top-K、Top-P、Softmax、随机采样等），流程碎片化严重。每个 Kernel 独立访问全局词表（vocab_size 级别），导致 HBM 加载次数线性膨胀；此外，重复惩罚阶段的掩码数据需通过 CPU-GPU 拷贝传输，引入额外同步开销。

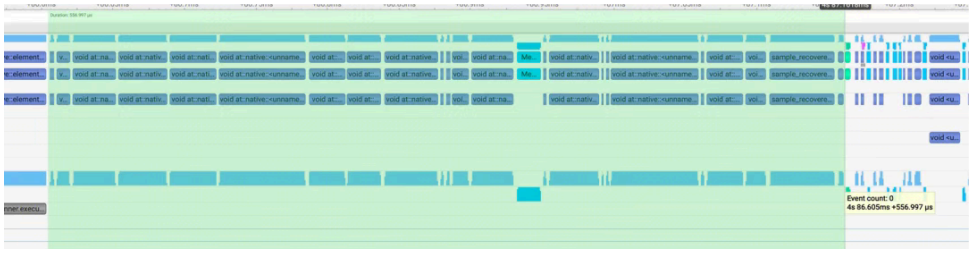
方案

将10余个零碎 Kernel融合为2个核心CUDA Kernel，并封装为单一fused_sampler算子。针对差异化业务场景提供更加精简专用内核，针对差异化业务场景（简单温度采样 / 完整采样），算子内自动适配调度专用内核，最大化 GPU 利用率。

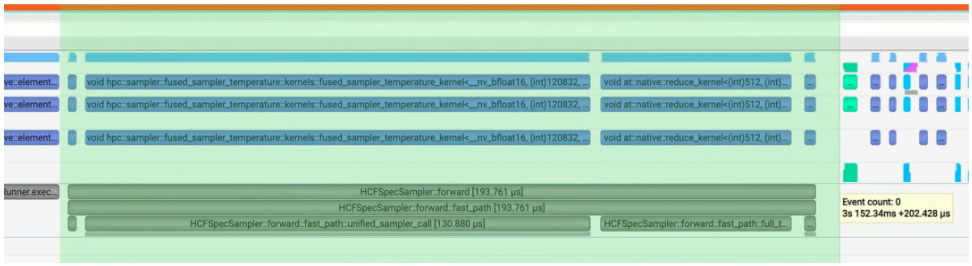
- **全词表单次加载**：将采样全流程所需的全局词表 GPU 读取压缩至 **1 次**，计算与访存充分掩盖。
- **GPU 闭环惩罚计算**：重复惩罚掩码在 GPU 内部完成，彻底消除 CPU-GPU 数据拷贝开销。
- **细粒度多 CTA 并行**：单请求拆分至多线程块并行执行，提升小 Batch 场景下的 SM 并发度。
- **局部堆归并 Top-K**：当 Max Top-K ≤ 64 时，采用局部堆归并替代全局阈值扫描或拒绝采样，避免全词表重复读取。
- **Top-K 与 Softmax 融合**：将 Top-K 归约与 Softmax 的 max/sum 计算合并，进一步削减访存与计算开销。

收益

融合前：



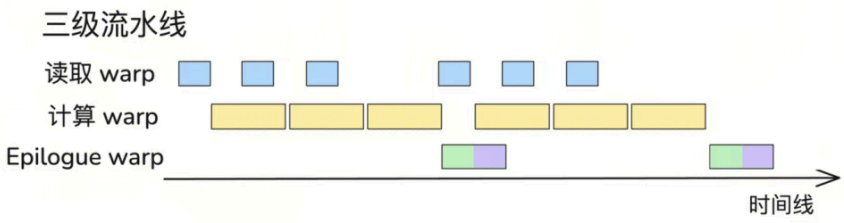
融合后：



相比 vLLM 与 FlashInfer 里的采样算子提升约 **5.5x、2.5x**

2.2.4 Gemm+Comm细粒度通算融合

针对 prefill TPSP 并行场景，我们实现 GEMM 与 ReduceScatter 的细粒度通算融合。SM 资源被显式划分为计算 SM（执行矩阵乘）与通信 SM（执行 RS 搬运）两类角色：计算 SM 每产出一个 Tile 即落盘至本地 Buffer，并通过信号量通知通信 SM 对就绪分片立即发起 RS，实现 Tile 级计算与通信重叠。



在传统 Load Warp 与 MMA Warp 之外，特化出专职 **Epilogue Warp**，形成 `Load → MMA → Epilogue` 三级流水：

1. **Load Warp**：异步预取下一轮 Tile
2. **MMA Warp**：累加完毕后仅写回 SMEM，立即进入下一轮计算
3. **Epilogue Warp**：异步取出 SMEM 结果，完成 Quant/Scale 等后处理并写回 HBM，最后触发通信 SM 就绪信号

性能收益

矩阵形状	分段耗时			端到端耗时		加速比
(M, N, K)	GEMM	RS	串行	本方案	覆盖率	vs 串行
(8k, 4096, 1024)	257.74	250.93	560.12	316.63	76.5%	1.77×
(16k, 4096, 1024)	512.04	480.82	1,096.59	604.98	80.7%	1.81×
(32k, 4096, 1024)	1,016.46	945.63	1,962.30	1,162.79	84.5%	1.69×
(64k, 4096, 1024)	2,026.13	1,846.68	3,872.58	2,307.68	84.8%	1.68×

**本能力由腾讯混元AI Infra团队与腾讯网络平台部联合优化打磨。*

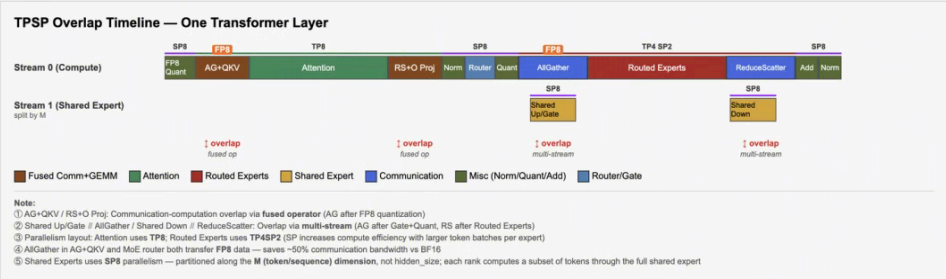
2.3 并行策略优化

2.3.1 Prefill并行策略：TPSP

Hy3 preview 模型上纯 TP8 方案会引入三重代价：

1. **冗余计算**：Elementwise / Router 等 token-wise 算子沿完整序列维度在各卡重复执行
2. **通信过重**：AllReduce 在 8 卡间交换全量数据，通信开销显著
3. **算子畸形**：MoE Grouped GEMM 被沿 hidden 维切至极窄 shape，计算效率急剧退化

方案



在保持单机 8 卡部署与模型精度不变的前提下，通过 **SP 拆分 + 通算融合 + 通信量化 + 并行模式调整** 四项技术组合，系统性压缩 TTFT

性能收益

场景	优化前 TTFT	优化后 TTFT	节省	降幅
Prefill 16k	764 ms	536 ms	−228 ms	−29.9%
Prefill 32k	1885 ms	1424 ms	−461 ms	−24.5%

2.3.2 Decode并行策略：DP+EP

问题

Hy3 preview 在单机部署时面临存算双重瓶颈：

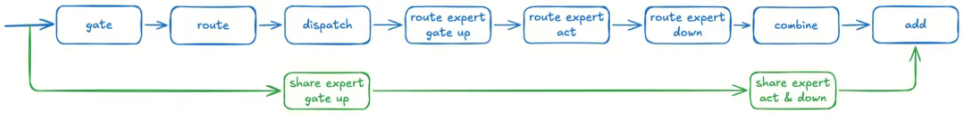
1. 显存被权重占用将近一半，挤压 KV Cache 空间，直接制约最大并发数
2. 在小 Batch 场景下，MoE Grouped GEMM 算力强度（Arithmetic Intensity）低，严重受限于访存带宽（Memory-bound）

方案

采用 Attention DP + MoE EP 的跨节点混合并行架构。通过增加专家并行度（EP Size）实现权重的多机分布式存储，以此腾出显存空间转产为 KV Cache 吞吐。同时，跨节点聚合 Batch Size 使 Grouped GEMM 进入 Compute-bound 区域，最大化 Tensor Core 利用率。

- 自研 HPC Kenrel，包含 gate，route，group gemm ， count and gather ， combine，在 Hopper卡上取得 sota 性能
- 长序列 Attn DPTP 混合策略，大幅降低 dp 负载不均衡所有带来的影响，而只需承担少量机内通信的额外开销

- 异步专家负载均衡 (Async EPLB)，利用 NCCL P2P 异步执行权重重排。将每步仅一层权重的通信逻辑隐藏于前向计算之后，实现权重重排与 Decode 计算完全重叠，消除通信干预。
- shared expert 拆分与 dispatch ，combine 并行，通信与计算 overlap



性能收益：端到端吞吐提升 **15.7 ～ 44.7%**

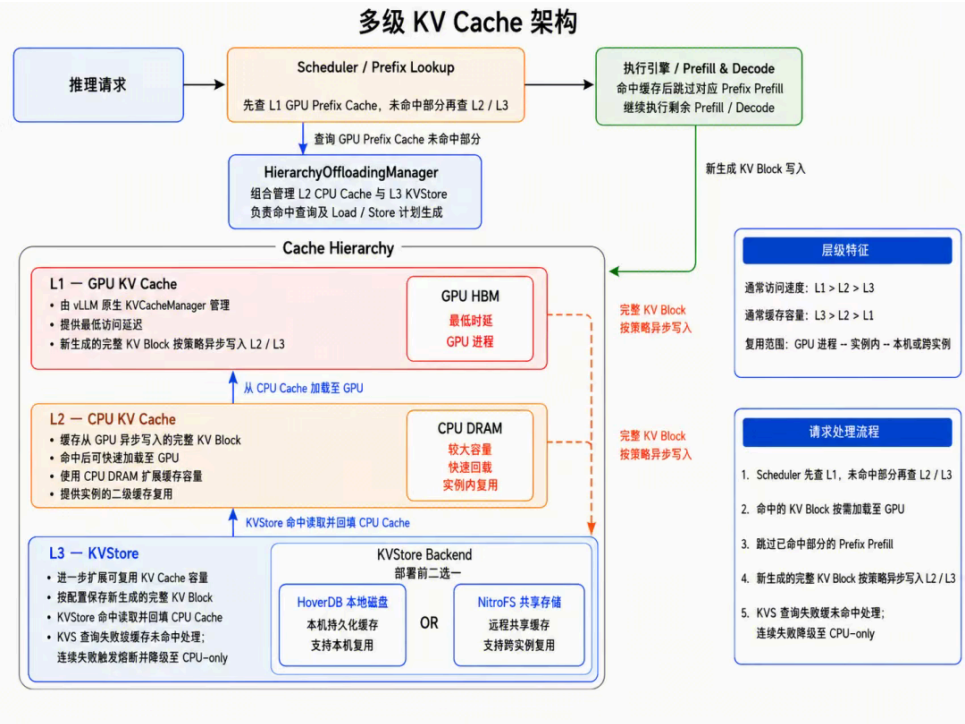
2.4 多级缓存

背景与动机

Agent、Coding等场景中存在大量长上下文、多轮对话和可复用公共前缀，Prefill 计算开销直接影响 TTFT 与整体吞吐。然而 Prefix Cache 如果仅依赖 GPU 显存，面临四重瓶颈：

1. **容量受限**：权重与运行时占用后，可用于缓存的显存空间极为有限
2. **淘汰加速**：长文请求产生的大体量 KV Cache 加速已有缓存淘汰
3. **重复计算**：缓存淘汰后相同前缀需重新 Prefill，算力浪费严重
4. **跨实例不可复用**：单机缓存在实例迁移、扩缩容、跨节点调度时完全失效

方案



构建 **GPU** → **CPU** → **KVStore** 三级缓存体系，将 KV Cache 从单一显存短期缓存扩展为可分层存储、按需加载、跨请求复用的多级架构。在不增加 GPU 显存占用的前提下，显著扩大有效缓存容量，降低重复 Prefill 概率。

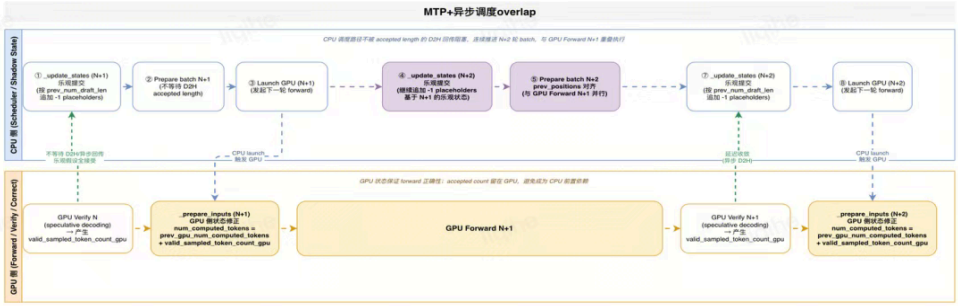
调度流程：请求进入时按 L1→L2→L3 顺序查询可复用前缀，命中后按需加载回 GPU 并跳过对应 Prefill；新生成的完整 Block 根据策略异步下沉至 L2/L3，供后续请求复用。

2.5 MTP和异步调度优化

问题

传统异步调度基于"每轮稳定生成 1 个 token"的假设，在 GPU 计算时让 CPU 提前准备下一轮输入，从而掩盖 CPU 耗时。然而，多层 MTP 引入了动态接收长度——下一轮的序列长度、位置编码及 KV Cache 映射均强依赖验证结果。传统做法需在 GPU Forward 验证结束后强制同步，将结果拷回 CPU 再准备下一轮输入，导致 CPU 准备阶段只能与 MTP 层 Forward 重叠；而 MTP 层计算极快，无法充分掩盖 CPU 耗时。

方案



核心思想： 解除 CPU 对真实接收长度的同步依赖——数据准备阶段一律按最大接收长度更新状态并组装下一轮输入；在下一轮实际计算前，再以上一轮的真实验证结果修正计算所依赖的关键值。由此，CPU 可提前一轮完成准备与 Launch，无需阻塞等待 GPU 计算结果。

收益：减少decode间 5～ 10ms CPU气泡， 端到端提升 **10%～20%** 性能

2.6 量化压缩

2.6.1 量化

问题

模型规模持续增长带来显存瓶颈与访存带宽压力，已成为部署落地的核心约束。直接应用 W4A8 量化和Attn FP8量化虽能大幅压缩模型体积，但权重的极低比特表示与激活中的离群值会严重放大量化误差，导致精度显著退化。

方案



在 **AngelSlim** 框架中构建 Hy3 preview 量化方案，通过 **"GPTQ 权重重建 + 激活平滑与旋转变换 + QAT 轻量化微调"** 三级联合优化，系统性消除Attn FP8 + W4A8 配置下的精度损失。

- **GPTQ 逐层权重重建**
  - 基于 Hessian 逆的逐层误差补偿，显著降低 INT4 权重量化引入的精度损失。
- **激活平滑（Smooth）**
  - 利用搜索得到的逐通道平滑因子，在不改变模型输出的前提下压缩激活离群值幅度，有效收窄数值动态范围。
- **Hadamard 在线旋转变换（Attention 专用）**
  - 对 Query/Key 量化前施加正交旋转，将集中于少数通道的离群值均匀打散，抑制量化误差；变换本身计算高效、可融合至推理 Kernel，几乎零额外开销。
- **QAT 轻量化微调**
  - 训练中模拟 W4A8 量化噪声，仅更新量化相关参数（如 scale/zero-point），使模型自适应学习最优量化配置。收敛迅速，训练成本极低。

性能收益

- 精度：多领域评测集上与 BF16 基线持平或差距 < 1%，精度无损。
- 性能：端到端吞吐提升**28%+**

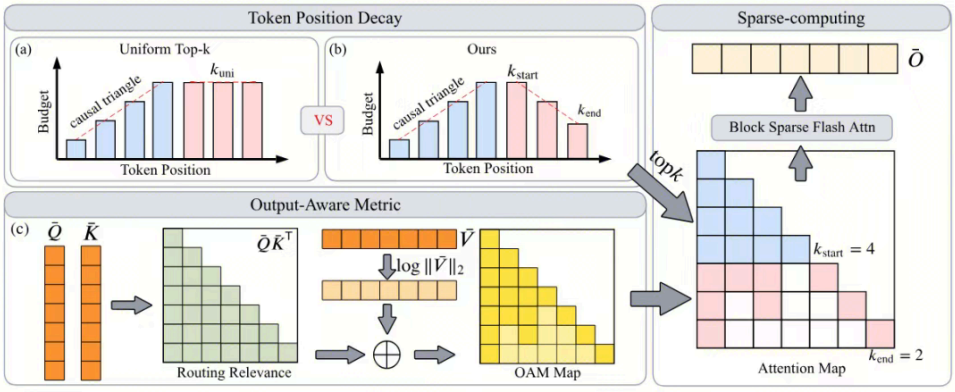
2.6.2 稀疏注意力

Hy3 preview 支持最大 256K 上下文，但标准自注意力的二次方复杂度导致 Prefill 阶段延迟和显存开销随序列长度急剧增长，成为制约 TTFT 的关键瓶颈。

为此，我们提出了 Stem 稀疏注意力算法，结合 HPC-BSA（Block Sparse Attention）算子，在仅使用 25% 计算预算的条件下实现接近稠密注意力的精度，将 128K 上下文下的 Prefill 延

迟降低了 **3.6 倍**。

核心思路：从因果注意力的信息流视角出发，重新审视"哪些 token 该保留、哪些该剪枝"，配合 HPC-BSA（Block Sparse Attention）算子，在仅使用 **25%**计算预算的条件下实现接近稠密注意力的精度。



Stem整体流程图

两大关键技术

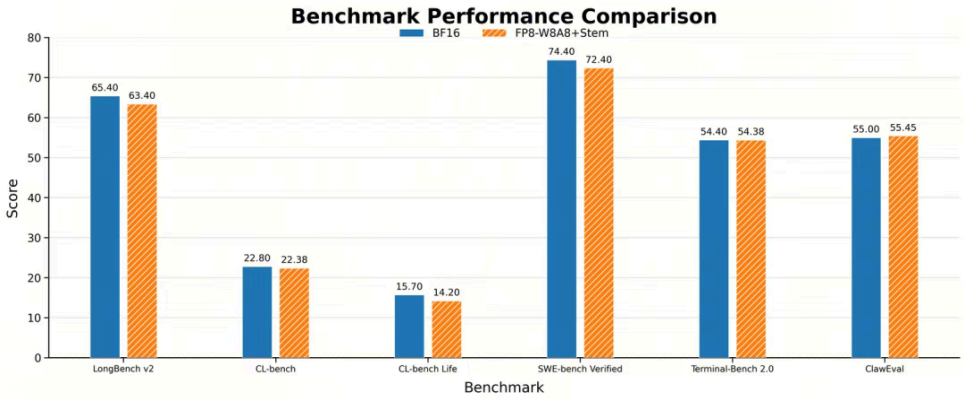
1. **Token Position-Decay（位置衰减策略）**
   - **洞察**：因果架构中，序列头部 token 参与了所有后续聚合计算，误差会逐层递归放大；尾部 token 影响仅局限于局部。
   - **做法**：将每个 query 位置的 Top-k 预算从头部的 `k_start` 线性衰减到尾部的 `k_end = μ · k_start`。头部关键 token 获得更大预算以保护递归依赖链，尾部冗余 token 被激进剪枝。
   - **效果**：总预算不变，仅通过重新分配即可显著提升精度。
2. **Output-Aware Metric（输出感知度量，OAM）**
   - **洞察**：传统方法仅基于注意力分数（QK^T）选择 token，但注意力分数只反映"路由概率"，并非实际"信息贡献"。高注意力但 Value 模长接近零的 token 对输出几乎无贡献。
   - **做法**：提出OAM度量公式 `M(i,j) = QK^T + β · max(0, log(‖V_j‖₂))`，将 Value 向量模长作为信号强度引入选择标准。
   - **优势**：对数变换使其可直接复用标准 Top-k 内核，无额外计算开销。

具体方案已集成至**AngelSlim**

收益

模型精度上，在LongBench v2、CL-bench、SWA等多个数据集上去的与密集注意力相当的精度水平。对比密集注意力，在128K上模型首字耗时提升**3.6倍**。

Speedup	16k	32k	64k	128k
FA2 BF16	1.00×	1.00×	1.00×	1.00×
FA3 BF16	1.27×	1.36×	1.42×	1.47×
FA3 FP8	1.45×	1.73×	2.02×	2.29×
Stem	1.50×	1.96×	2.68×	3.62×



三、后续工作

在长上下文场景下，经过一系列优化后仍面临显存瓶颈，为此我们正积极推进 **C4 与 W4 相关优化**，在确保精度无损的前提下，进一步提升推理吞吐能力。

针对超高吐字速度的需求，我们正在探索全新的并行投机解码方案——在保证接收率的同时，以更低的计算代价产出更多的投机 Token，有望实现吐字速率的大幅跃升。

与此同时，我们也在以下关键环节持续进行深入优化：调度和并行策略、PD 高效传输、多级缓存中心、跨机通信与流量控制等

此外，我们同步推进对其他硬件平台的适配与优化。凭借更优的硬件性价比，推理成本有望进一步降低，敬请期待后续的分享与成果发布。



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

688篇原创内容

公众号



阅读 1.3万

留言

写留言

还没有留言，来写首评